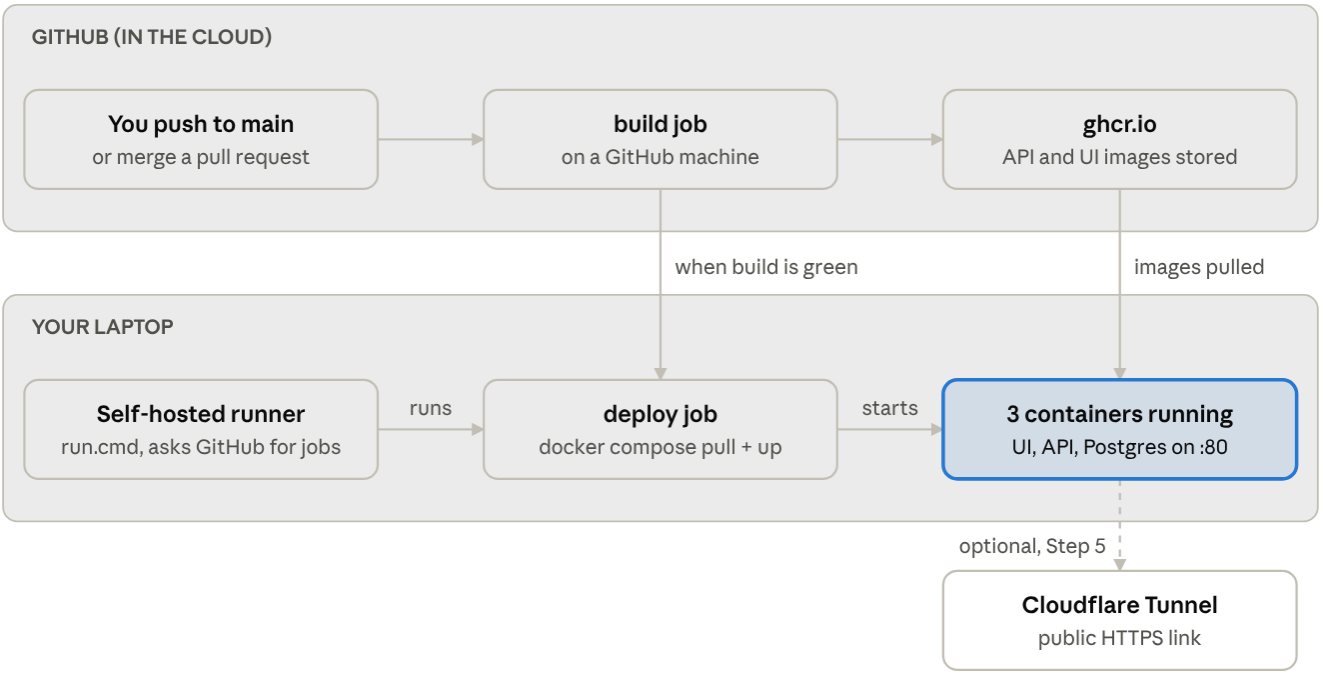
Deploy the URL Shortener on Your Laptop

Nov 6, 2026 · @Amit

How it works

Every push to `main` builds fresh images on GitHub and restarts the app on your laptop, with no server to rent. The trick is a **self-hosted runner**: a GitHub program on your laptop that connects out to GitHub and asks for work, so GitHub never has to reach in.

GitHub builds the images; your laptop runs them



deployment flow · GitHub to laptop

The `build` job runs on GitHub's machines; the `deploy` job runs on your laptop. Your laptop only downloads finished images, so it needs Docker but not Go or Node.

Before you start

You need four things in place. Check each one before Step 1.

- [ ] **A Windows laptop with Docker Desktop installed and running.** Run `docker version` in PowerShell; it should print both Client and Server.
- [ ] **The app working locally.** `docker compose up --build` in the `shortener` folder, then http://localhost works.
- [ ] **The code on GitHub, in a private repo,** with `api/go.sum` and `ui/package-lock.json` committed and `.env` not committed. Both Dockerfiles need the two lock files.

- [ ] **The `build` job passing** in the repo's **Actions** tab. It runs on GitHub's machines and needs no setup from you.

Keep the repo private. A self-hosted runner runs whatever the workflow says on your laptop, and GitHub advises against using one with a public repo, where anyone could open a pull request.

Step 1: Install the GitHub runner on your laptop

The runner is a small GitHub program. It asks GitHub for jobs over an outgoing connection, so your router never has to let anything in.

1. In your GitHub repo, open **Settings → Actions → Runners → New self-hosted runner**.
2. Choose **Windows** and **x64**. GitHub shows a list of commands with a one-time token already filled in.
3. Open **PowerShell**. Create a folder for the runner outside your code, for example `C:\actions-runner`, and `cd` into it.
4. Copy the **Download** commands from the GitHub page and run them one at a time. They download and unzip the runner.
5. Copy the **Configure** command, which starts with `./config.cmd --url https://github.com/...`, and run it.
6. Answer the questions it asks:
   - Runner group, name, labels, work folder: press **Enter** to accept each default.
   - "Run the runner as a service?": type **N** for now. You start it by hand in the next step.
7. Start the runner:

```
./run.cmd
```

8. Wait for **Listening for Jobs**. Keep this window open; closing it stops the runner.
9. Back on GitHub, **Settings → Actions → Runners** now shows your laptop as **Idle** with a green dot.

Next time, you only need steps 3 and 7: `cd C:\actions-runner`, then `./run.cmd`.

Step 2: Add the database password as a secret

On a laptop you need just one secret. `SERVER_HOST`, `SERVER_USER` and `SERVER_SSH_KEY` are only for deploying to a cloud server over SSH; delete them if you added them.

1. In your repo, open **Settings → Secrets and variables → Actions**.

2. Stay on the **Secrets** tab and click **New repository secret**.
3. Fill in:
   - **Name:** `DB_PASSWORD`
   - **Secret:** any password you choose, for example `MyDemoPass123`
4. Click **Add secret**.

This replaces the `.env` file. The workflow hands the password to Docker Compose directly, so no `.env` is needed for deployments.

Step 3: Update the workflow file

Replace the whole of `.github/workflows/deploy.yml` with this. The `build` job is unchanged; only `deploy` is new.

```
name: Build and Deploy
# The name shown in the Actions tab.

on:
  push:
    branches: [main]
# Runs on every push or merge into main.

jobs:
  build:
    runs-on: ubuntu-latest
    # Runs on a fresh GitHub machine, which is deleted afterwards.
    permissions:
      contents: read
      packages: write
      # Allowed to push images to GitHub Container Registry.
    steps:
      - uses: actions/checkout@v4
      # Downloads your code onto the GitHub machine.

      - name: Log in to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
          # GITHUB_TOKEN is created by GitHub for every run.

      - name: Build and push API image
        uses: docker/build-push-action@v6
```

```
        with:
          context: ./api
          # Uses api/Dockerfile; Go is compiled here.
          push: true
          tags: |
            ghcr.io/${{ github.repository_owner }}/shortener-api:latest
            ghcr.io/${{ github.repository_owner }}/shortener-api:${{
github.sha }}
          # "latest" is what gets deployed; the commit hash is kept for
rollback.

      - name: Build and push UI image
        uses: docker/build-push-action@v6
        with:
          context: ./ui
          # Uses ui/Dockerfile; React is built and packed into Nginx here.
          push: true
          tags: |
            ghcr.io/${{ github.repository_owner }}/shortener-ui:latest
            ghcr.io/${{ github.repository_owner }}/shortener-ui:${{
github.sha }}

  deploy:
    needs: build
    # Starts only after build succeeds.
    runs-on: self-hosted
    # Runs on YOUR laptop, through the runner from Step 1.
    env:
      DB_PASSWORD: ${{ secrets.DB_PASSWORD }}
      # The secret from Step 2; Docker Compose reads it from here.
      GH_OWNER: ${{ github.repository_owner }}
      # Your GitHub username, used in the image names.
    steps:
      - uses: actions/checkout@v4
      # Puts docker-compose.yml, nginx.conf and init.sql on your laptop.

      - name: Log in to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      # Lets your laptop download the private images.

      - name: Start containers
        run: |
```

```
          docker compose pull
          docker compose up -d --no-build
      # pull: download the new images.
      # up -d --no-build: replace only the changed containers, in the
background,
      # using the downloaded images instead of building them again.
```

If your GitHub username has capital letters, the push to `ghcr.io` fails. Replace every `${{ github.repository_owner }}` with your username in lowercase, five places in total.

Step 4: Deploy and check

1. Make sure **Docker Desktop is running** and the runner window shows **Listening for Jobs**.
2. Stop the copy you started by hand earlier, because it also uses port 80. In your own `shortener` folder:

```
docker compose down
```

3. Commit the new `deploy.yml` and get it into `main`, either by merging a pull request or with:

```
git checkout main
git pull origin main
git merge my-branch
git push origin main
```

4. Open the repo's **Actions** tab and click the newest **Build and Deploy** run.
5. `build` runs first, on GitHub. When it turns green, `deploy` starts, and you'll see it appear in the runner window on your laptop.
6. When both are green, check the containers:

```
docker ps
```

You should see three containers, for the UI, API and database, all **Up**.

7. Open **http://localhost** and shorten a URL. The app is now deployed by GitHub Actions.

From now on, every push to `main` rebuilds the images and replaces the changed containers on your laptop. Your saved links survive, because they live in the `pgdata` volume, not in a container.

Step 5: Share it with others (Cloudflare Tunnel)

After Step 4, only you can open the app. `localhost` means "this computer", and your home router blocks requests coming in from the internet. A Cloudflare Tunnel works like the runner: your laptop connects **out** to Cloudflare, and Cloudflare gives you a public HTTPS link that leads back down that connection.

1. Install it in PowerShell, then close and reopen PowerShell:

```
winget install --id Cloudflare.cloudflared
```

2. Check that http://localhost works on your laptop.
3. Start the tunnel:

```
cloudflared tunnel --url http://localhost:80
```

4. In the output, find the link ending in `.trycloudflare.com`, for example `https://blue-river-cat.trycloudflare.com`.
5. Open it on your phone using mobile data, not your Wi-Fi, to prove it works from outside.
6. Send the link to anyone. Stop sharing with **Ctrl+C**.

This quick mode needs no Cloudflare account, but it gives a **new random link every time** you start it. A fixed link needs a free Cloudflare account and a domain of your own.

Someone on your **same Wi-Fi** can skip the tunnel: run `ipconfig`, find your laptop's IPv4 address (like `192.168.1.5`), and they open `http://192.168.1.5`. If it doesn't load, Windows Firewall is blocking port 80.

If something goes wrong

What you see	Why	Fix
`deploy` stays on "Waiting for a runner to pick up this job"	The runner isn't running	`cd C:\actions-runner`, then `./run.cmd`
`build` fails with "denied" on push	GitHub username has capital letters	Use your lowercase username in `deploy.yml` (Step 3)
`deploy` fails: "Cannot connect to the Docker daemon"	Docker Desktop is closed	Start Docker Desktop, then **Re-run failed jobs**

What you see	Why	Fix
`deploy` fails: "port is already allocated"	Something else uses port 80, often your manual copy	`docker compose down` in your own folder, then re-run
`api` container keeps restarting	Database password missing or wrong	Check the `DB_PASSWORD` secret is under **Secrets**, not **Variables**
http://localhost doesn't load	A container isn't up	`docker ps`, then `docker logs <container-name>`

To re-run a failed deployment without pushing again: **Actions** tab → the failed run → **Re-run jobs → Re-run failed jobs**.

Limits of a laptop server

- The app is online only while the laptop is **on**, Docker Desktop is running and, for deployments, `run.cmd` is open.
- The tunnel link changes every time you restart it.
- One laptop is fine for learning and demos. For an app that stays online for real users, move it to a cloud VM; the `build` job stays the same and only `deploy` changes back to SSH.